Rapport TP NoSQL Security

Séance 2 — Modélisation NoSQL, Requêtes Avancées & Index

Plateforme SOC-lite

Étudiant : Daoui asma

2ème année Cybersécurité

5 avril 2026

Table des matières

1 Introduction **3**
1.1 Environnement technique . . . 3
1.2 Commandes d'initialisation . . . 3

2 TP 1 — Modélisation et Insertion des données **4**
2.1 Collection users (Embed : rôles) . . . 4
2.2 Collection security_logs (Time-series) . . . 4
2.3 Collection incidents (avec références sécurisées) . . . 6
2.4 Vérification . . . 7

3 TP 2 — Manipulation des relations **8**
3.1 Ajout d'un commentaire (Embed avec $push) . . . 8
3.2 Ajout de nouveaux logs . . . 8
3.3 Création d'un incident lié (méthode sécurisée) . . . 9
3.4 Many-to-Many : plusieurs analystes . . . 9

4 TP 3 — Requêtes SOC avancées **10**
4.1 Requêtes guidées . . . 10
4.2 Exercices autonomes . . . 10

5 TP 4 — Aggregation Pipeline (Statistiques SOC) **12**

6 TP 5 — Index et optimisation **13**

7 TP 6 — Sécurité data-level **14**
7.1 Identification de la sur-collecte . . . 14
7.2 Masquage des emails (PII) . . . 14
7.3 Projection sûre pour un viewer . . . 14
7.4 Checklist Sécurité Data . . . 14

8 Devoir maison — Mini dataset IDS **16**
8.1 Insertion des données . . . 16
8.2 Requêtes IDS . . . 16

9 Checkpoints — Questions de réflexion **18**
9.1 Checkpoint 1 18
9.2 Checkpoint 2 18
9.3 Checkpoint 3 18

10 Commandes de nettoyage **19**

11 Conclusion **19**

1 Introduction

Ce rapport documente l'ensemble des travaux pratiques réalisés sur MongoDB dans le cadre du module NoSQL Security. L'objectif est de construire une plateforme SOC-lite permettant la gestion d'incidents de sécurité, l'analyse de logs et l'implémentation de bonnes pratiques de sécurité data-level.

1.1 Environnement technique

- MongoDB 7+ avec Docker
- Port d'écoute : 27020 (redirection 27020 :27017)
- Base de données : `soc_lite`
- Collections : `users`, `security_logs`, `incidents`, `ids_alerts`

1.2 Commandes d'initialisation

```
docker rm -f mongo-soc 2>/dev/null
docker run -d --name mongo-soc -p 27020:27017 mongo:7
docker exec -it mongo-soc mongosh --port 27017
```

```
use soc_lite
```

2 TP 1 — Modélisation et Insertion des données

2.1 Collection users (Embed : rôles)

```
db.users.insertMany([
  {
    username: "alice_analyst",
    full_name: "Alice Dupont",
    email: "alice@soc.local",
    roles: ["analyst", "viewer"],
    team: "Blue Team",
    created_at: new Date("2025-01-15"),
    last_login: new Date("2025-03-01T08:30:00Z"),
    is_active: true
  },
  {
    username: "bob_senior",
    full_name: "Bob Martin",
    email: "bob@soc.local",
    roles: ["senior_analyst", "incident_manager"],
    team: "Blue Team",
    created_at: new Date("2024-06-10"),
    last_login: new Date("2025-03-03T14:12:00Z"),
    is_active: true
  },
  {
    username: "charlie_admin",
    full_name: "Charlie Leroy",
    email: "charlie@soc.local",
    roles: ["admin"],
    team: "SOC Management",
    created_at: new Date("2024-01-05"),
    last_login: new Date("2025-02-28T18:00:00Z"),
    is_active: true
  },
  {
    username: "diana_junior",
    full_name: "Diana Nour",
    email: "diana@soc.local",
    roles: ["viewer"],
    team: "Blue Team",
    created_at: new Date("2025-02-01"),
    last_login: new Date("2025-03-03T09:00:00Z"),
    is_active: true
  }
])
```

2.2 Collection security_logs (Time-series)

```
db.security_logs.insertMany([
```

```
  {
    timestamp: new Date("2025-03-03T10:05:22Z"),
    event_type: "login_failed",
    src_ip: "192.168.1.105", dst_ip: "10.0.0.2",
    src_port: 44821, dst_port: 22,
    protocol: "SSH", user_attempted: "root",
    device_id: "srv-web-01", severity: "medium",
    raw_log: "Failed password for root from 192.168.1.105 port 44821 ssh2"
  },
  {
    timestamp: new Date("2025-03-03T10:05:23Z"),
    event_type: "login_failed",
    src_ip: "192.168.1.105", dst_ip: "10.0.0.2",
    src_port: 44822, dst_port: 22,
    protocol: "SSH", user_attempted: "root",
    device_id: "srv-web-01", severity: "medium",
    raw_log: "Failed password for root from 192.168.1.105 port 44822 ssh2"
  },
  {
    timestamp: new Date("2025-03-03T10:05:24Z"),
    event_type: "login_failed",
    src_ip: "192.168.1.105", dst_ip: "10.0.0.2",
    src_port: 44823, dst_port: 22,
    protocol: "SSH", user_attempted: "admin",
    device_id: "srv-web-01", severity: "medium",
    raw_log: "Failed password for admin from 192.168.1.105 port 44823 ssh2"
  },
  {
    timestamp: new Date("2025-03-03T10:12:00Z"),
    event_type: "login_success",
    src_ip: "10.0.1.50", dst_ip: "10.0.0.5",
    src_port: 52100, dst_port: 443,
    protocol: "HTTPS", user_attempted: "alice",
    device_id: "siem-console", severity: "info",
    raw_log: "Successful login for alice from 10.0.1.50"
  },
  {
    timestamp: new Date("2025-03-03T11:30:45Z"),
    event_type: "port_scan",
    src_ip: "172.16.0.88", dst_ip: "10.0.0.0/24",
    src_port: null, dst_port: null,
    protocol: "TCP", user_attempted: null,
    device_id: "ids-sensor-02", severity: "high",
    raw_log: "Nmap SYN scan detected from 172.16.0.88 targeting 10.0.0.0/24"
  },
  {
    timestamp: new Date("2025-03-03T14:22:10Z"),
```

```
    event_type: "malware_detected",
    src_ip: "192.168.2.30", dst_ip: "10.0.0.10",
    src_port: 80, dst_port: 8080,
    protocol: "HTTP", user_attempted: null,
    device_id: "endpoint-pc-042", severity: "critical",
    raw_log: "Trojan.GenericKD.46543210 detected in /tmp/update.exe"
  },
  {
    timestamp: new Date("2025-03-03T15:00:00Z"),
    event_type: "login_failed",
    src_ip: "10.0.1.200", dst_ip: "10.0.0.5",
    src_port: 61234, dst_port: 443,
    protocol: "HTTPS", user_attempted: "charlie",
    device_id: "siem-console", severity: "low",
    raw_log: "Failed login for charlie - wrong password"
  },
  {
    timestamp: new Date("2025-03-03T16:45:00Z"),
    event_type: "data_exfiltration_suspect",
    src_ip: "10.0.0.10", dst_ip: "203.0.113.50",
    src_port: 443, dst_port: 443,
    protocol: "HTTPS", user_attempted: null,
    device_id: "firewall-main", severity: "critical",
    raw_log: "Unusual outbound transfer 2.3GB to 203.0.113.50"
  }
])
```

2.3 Collection incidents (avec références sécurisées)

```
// 1. R cup rer les IDs des logs SSH (m thode s curis e)
let logIds = []
db.security_logs.find({ src_ip: "192.168.1.105" }).limit(100).forEach(d => logIds.push(d._id))

// 2. R cup rer les IDs des logs malware
let logIds2 = []
db.security_logs.find({
  event_type: { $in: ["malware_detected","data_exfiltration_suspect"] }
}).limit(100).forEach(d => logIds2.push(d._id))

// 3. Insertion des incidents
db.incidents.insertMany([
  {
    incident_id: "INC-2025-001",
    title: "Brute force SSH sur srv-web-01",
    description: "Multiples tentatives SSH   choues   depuis 192.168.1.105 ciblant root et admin.",
    severity: "high", status: "investigating",
```

```
    created_at: new Date("2025-03-03T10:30:00Z"),
    updated_at: new Date("2025-03-03T11:00:00Z"),
    assigned_to: "alice_analyst",
    tags: ["brute-force", "ssh", "external"],
    related_log_ids: logIds,
    comments: [
      { author: "alice_analyst",
        text: "3 tentatives en 2s depuis la m me IP. Pattern brute
   -force confirm .",
        timestamp: new Date("2025-03-03T10:35:00Z") }
    ],
    mitre_tactics: ["TA0006"],
    mitre_techniques: ["T1110.001"]
  },
  {
    incident_id: "INC-2025-002",
    title: "Malware d tect  sur endpoint-pc-042",
    description: "Trojan d tect  dans /tmp/update.exe.
   Potentielle exfiltration en cours.",
    severity: "critical", status: "active",
    created_at: new Date("2025-03-03T14:30:00Z"),
    updated_at: new Date("2025-03-03T16:50:00Z"),
    assigned_to: "bob_senior",
    tags: ["malware", "exfiltration", "endpoint"],
    related_log_ids: logIds2,
    comments: [
      { author: "bob_senior",
        text: "Isolement r seau du poste effectu . Analyse
   forensique en cours.",
        timestamp: new Date("2025-03-03T14:45:00Z") }
    ],
    mitre_tactics: ["TA0010","TA0011"],
    mitre_techniques: ["T1041","T1071.001"]
  }
])
```

2.4 Vérification

```
db.users.countDocuments()          //     4
db.security_logs.countDocuments()  //     8
db.incidents.countDocuments()      //     2
```

3 TP 2 — Manipulation des relations

3.1 Ajout d'un commentaire (Embed avec $push)

```
db.incidents.updateOne(
  { incident_id: "INC-2025-001" },
  {
    $push: { comments: {
      author: "diana_junior",
      text: "L'IP 192.168.1.105 appartient au sous-r seau visiteur
    . Aucun compte legitime.",
      timestamp: new Date()
    }},
    $set: { updated_at: new Date() }
  }
)

// Verification
db.incidents.findOne(
  { incident_id: "INC-2025-001" },
  { comments: 1, incident_id: 1 }
)
```

3.2 Ajout de nouveaux logs

```
db.security_logs.insertMany([
  {
    timestamp: new Date("2025-03-03T17:00:00Z"),
    event_type: "login_failed",
    src_ip: "10.0.1.200", dst_ip: "10.0.0.5",
    src_port: 61300, dst_port: 443,
    protocol: "HTTPS", user_attempted: "charlie",
    device_id: "siem-console", severity: "medium",
    raw_log: "Failed login for charlie - account locked"
  },
  {
    timestamp: new Date("2025-03-03T17:00:05Z"),
    event_type: "login_failed",
    src_ip: "10.0.1.200", dst_ip: "10.0.0.5",
    src_port: 61305, dst_port: 443,
    protocol: "HTTPS", user_attempted: "charlie",
    device_id: "siem-console", severity: "medium",
    raw_log: "Failed login for charlie - account locked"
  },
  {
    timestamp: new Date("2025-03-03T17:01:00Z"),
    event_type: "privilege_escalation",
    src_ip: "10.0.1.200", dst_ip: "10.0.0.5",
    src_port: 61310, dst_port: 443,
    protocol: "HTTPS", user_attempted: "charlie",
```

```
    device_id: "siem-console", severity: "critical",
    raw_log: "Privilege escalation attempt detected for charlie"
  }
])
```

3.3 Création d'un incident lié (méthode sécurisée)

```
// R cup ration des IDs (methode robuste)
let charlieLogIds = []
db.security_logs.find({
  src_ip: "10.0.1.200",
  timestamp: { $gte: new Date("2025-03-03T17:00:00Z") }
}).limit(100).forEach(d => charlieLogIds.push(d._id))

// Creation de l'incident
db.incidents.insertOne({
  incident_id: "INC-2025-003",
  title: "Tentative d'escalade de privileges - charlie",
  description: "Echecs de connexion suivis d'une tentative d'escalade depuis 10.0.1.200.",
  severity: "critical", status: "new",
  created_at: new Date(), updated_at: new Date(),
  assigned_to: "alice_analyst",
  tags: ["privilege-escalation", "internal", "account-lockout"],
  related_log_ids: charlieLogIds,
  comments: [],
  mitre_tactics: ["TA0004"],
  mitre_techniques: ["T1078"]
})

// Verification de coherence
let inc3 = db.incidents.findOne({ incident_id: "INC-2025-003" })
db.security_logs.find({ _id: { $in: inc3.related_log_ids } }).count() //     3
```

3.4 Many-to-Many : plusieurs analystes

```
db.incidents.updateOne(
  { incident_id: "INC-2025-002" },
  { $addToSet: { assigned_analysts: { $each: ["bob_senior","alice_analyst"] } } }
)
```

4 TP 3 — Requêtes SOC avancées

4.1 Requêtes guidées

```
// R1 - Logs de severite "critical"
db.security_logs.find({ severity: "critical" })

// R2 - Logs entre 10h et 12h, tries
db.security_logs.find({
  timestamp: { $gte: new Date("2025-03-03T10:00:00Z"),
               $lte: new Date("2025-03-03T12:00:00Z") }
}).sort({ timestamp: 1 })

// R3 - Echecs login avec projection minimale
db.security_logs.find(
  { event_type: "login_failed" },
  { src_ip: 1, user_attempted: 1, timestamp: 1, _id: 0 }
)

// R4 - Les 3 derniers logs
db.security_logs.find().sort({ timestamp: -1 }).limit(3)

// R5 - IP specifique OU severite critical ($or)
db.security_logs.find({
  $or: [ { src_ip: "192.168.1.105" }, { severity: "critical" } ]
})

// R6 - Incidents actifs avec commentaires
db.incidents.find(
  { status: { $in: ["active","investigating"] } },
  { incident_id: 1, title: 1, "comments.author": 1, "comments.text": 1, _id: 0 }
)
```

4.2 Exercices autonomes

```
// E1 - Utilisateurs actifs du Blue Team
db.users.find(
  { team: "Blue Team", is_active: true },
  { username: 1, roles: 1, _id: 0 }
)

// E2 - Logs du device "siem-console", recents d'abord, max 5
db.security_logs.find({ device_id: "siem-console" })
  .sort({ timestamp: -1 }).limit(5)

// E3 - Compter les login_failed
db.security_logs.countDocuments({ event_type: "login_failed" })  // 5

```

```
// E4 - Incidents d'alice_analyst
db.incidents.find(
  { assigned_to: "alice_analyst" },
  { title: 1, severity: 1, _id: 0 }
)
```

5 TP 4 — Aggregation Pipeline (Statistiques SOC)

```
// A1 - Top IPs sources
db.security_logs.aggregate([
  { $group: { _id: "$src_ip", count: { $sum: 1 } } },
  { $sort: { count: -1 } },
  { $limit: 5 }
])

// A2 - Echecs de login par utilisateur
db.security_logs.aggregate([
  { $match: { event_type: "login_failed" } },
  { $group: {
      _id: "$user_attempted",
      failures: { $sum: 1 },
      last_attempt: { $max: "$timestamp" }
  }},
  { $sort: { failures: -1 } }
])

// A3 - Repartition des severites
db.security_logs.aggregate([
  { $group: { _id: "$severity", count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])

// A4 - Alertes par device (high/critical)
db.security_logs.aggregate([
  { $match: { severity: { $in: ["high","critical"] } } },
  { $group: {
      _id: "$device_id",
      alert_count: { $sum: 1 },
      event_types: { $addToSet: "$event_type" }
  }},
  { $sort: { alert_count: -1 } }
])

// Exercice - Evenements par heure le 3 mars
db.security_logs.aggregate([
  { $match: { timestamp: {
      $gte: new Date("2025-03-03T00:00:00Z"),
      $lt: new Date("2025-03-04T00:00:00Z")
  }}},
  { $group: {
      _id: { $dateToString: { format: "%H:00", date: "$timestamp" } },
      count: { $sum: 1 }
  }},
  { $sort: { _id: 1 } }
])
```

6 TP 5 — Index et optimisation

```
// Etape 1 - Sans index (COLLSCAN)
db.security_logs.find({ src_ip: "192.168.1.105" }).explain("executionStats")

// Etape 2 - Creer un index simple
db.security_logs.createIndex({ src_ip: 1 })

// Etape 3 - Verifier l'amelioration (IXSCAN)
db.security_logs.find({ src_ip: "192.168.1.105" }).explain("executionStats")

// Etape 4 - Index compose
db.security_logs.createIndex({ timestamp: 1, src_ip: 1 })

// Test avec requete temporelle + IP
db.security_logs.find({
  timestamp: { $gte: new Date("2025-03-03T10:00:00Z"),
               $lte: new Date("2025-03-03T11:00:00Z") },
  src_ip: "192.168.1.105"
}).explain("executionStats")

// Etape 5 - Lister les index
db.security_logs.getIndexes()

// Etape 6 - Prefix rule (src_ip seul utilise src_ip_1)
db.security_logs.find({ src_ip: "10.0.1.200" }).explain("executionStats")
```

7 TP 6 — Sécurité data-level

7.1 Identification de la sur-collecte

```
db.security_logs.findOne({ event_type: "malware_detected" })
// Le champ raw_log contient des chemins fichiers internes sensibles
```

7.2 Masquage des emails (PII)

```
db.users.updateMany({}, [
  { $set: { email_masked: {
      $concat: [
        { $substr: ["$email", 0, 1] },
        "***@",
        { $arrayElemAt: [{ $split: ["$email","@"] }, 1] }
      ]
  }}}
])

db.users.find({}, { username: 1, email: 1, email_masked: 1, _id: 0 })
```

7.3 Projection sûre pour un viewer

```
// Pour un viewer : pas d'email complet, pas de raw_log
db.users.find(
  { is_active: true },
  { username: 1, team: 1, roles: 1, email_masked: 1, _id: 0 }
)

db.security_logs.find(
  { severity: "critical" },
  { raw_log: 0 }
)
```

7.4 Checklist Sécurité Data

```
/*
## Checklist Securite Data - SOC-lite

1. [X] Perimetre fonctionnel par collection : users, security_logs, incidents
2. [X] PII identifies : email, full_name
3. [X] PII masques pour acces non privilegies : email -> email_masked
4. [X] Projections definies par role : viewer sans raw_log ni email complet
```

```
5. [X] Champs haute sensibilite (raw_log) : acces restreint
6. [X] Denormalisation ne duplique pas de PII : OK
7. [X] Index ne revelent pas de patterns sensibles : src_ip indexe
   mais necessaire
8. [X] Pas de sur-collecte : raw_log limite aux analystes seniors
*/
```

8 Devoir maison — Mini dataset IDS

8.1 Insertion des données

```
db.ids_alerts.insertMany([
  { timestamp: new Date("2025-03-04T08:00:00Z"), rule_id: "SID-1001",
    rule_name: "SQL Injection Attempt", src_ip: "203.0.113.15",
    dst_ip: "10.0.0.5", severity: "high", action: "alert", sensor: "ids-dmz-01" },
  { timestamp: new Date("2025-03-04T08:05:00Z"), rule_id: "SID-1001",
    rule_name: "SQL Injection Attempt", src_ip: "203.0.113.15",
    dst_ip: "10.0.0.5", severity: "high", action: "alert", sensor: "ids-dmz-01" },
  { timestamp: new Date("2025-03-04T09:00:00Z"), rule_id: "SID-2050",
    rule_name: "DNS Tunneling Detected", src_ip: "10.0.2.100",
    dst_ip: "8.8.8.8", severity: "critical", action: "block", sensor: "ids-internal-01" },
  { timestamp: new Date("2025-03-04T10:30:00Z"), rule_id: "SID-3001",
    rule_name: "Suspicious Outbound Connection", src_ip: "10.0.0.10",
    dst_ip: "198.51.100.77", severity: "medium", action: "alert", sensor: "ids-internal-01" },
  { timestamp: new Date("2025-03-04T11:00:00Z"), rule_id: "SID-1001",
    rule_name: "SQL Injection Attempt", src_ip: "203.0.113.15",
    dst_ip: "10.0.0.5", severity: "high", action: "drop", sensor: "ids-dmz-01" }
])
```

8.2 Requêtes IDS

```
// 1. Alertes severity "high" ou "critical"
db.ids_alerts.find({ severity: { $in: ["high", "critical"] } })

// 2. Alertes SID-1001, triees par timestamp
db.ids_alerts.find({ rule_id: "SID-1001" }).sort({ timestamp: 1 })

// 3. Nombre d'alertes par rule_id
db.ids_alerts.aggregate([
  { $group: { _id: "$rule_id", count: { $sum: 1 } } }
])

// 4. Alertes 8h-10h, projection minimale
db.ids_alerts.find(
  { timestamp: { $gte: new Date("2025-03-04T08:00:00Z"),
                 $lte: new Date("2025-03-04T10:00:00Z") } },
```

```
  { src_ip: 1, rule_name: 1, timestamp: 1, _id: 0 }
)

// 5. Les 2 alertes les plus recentes
db.ids_alerts.find().sort({ timestamp: -1 }).limit(2)

// 6. Top IPs sources
db.ids_alerts.aggregate([
  { $group: { _id: "$src_ip", count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])

// 7. Alertes action "block" ou "drop"
db.ids_alerts.find({ action: { $in: ["block", "drop"] } })

// 8. Nombre d'alertes par sensor
db.ids_alerts.aggregate([
  { $group: { _id: "$sensor", count: { $sum: 1 } } }
])

// 9. Alertes critiques du reseau interne
db.ids_alerts.find({
  severity: "critical",
  src_ip: { $regex: /^10\./ }
})

// 10. Alerte la plus ancienne
db.ids_alerts.find().sort({ timestamp: 1 }).limit(1)
```

9 Checkpoints — Questions de réflexion

9.1 Checkpoint 1

— **Q1 :** Quel est le risque principal d'un document dépassant 16 Mo ?
R : MongoDB refuse l'insertion/mise à jour. Le document est rejeté.

— **Q2 :** 150 000 logs liés à un incident : Embed ou Reference ? Pourquoi ?
R : Reference. 150 000 logs dépasseraient la taille limite (16 Mo), ralentiraient les lectures, et exposeraient les logs bruts à quiconque accède à l'incident.

— **Q3 :** Citez un exemple d'embed et un de reference dans notre modèle.
R : Embed : commentaires dans incidents, rôles dans users. Reference : related_log_ids vers security_logs.

9.2 Checkpoint 2

— **Q1 :** Écrivez la requête : logs SSH des 24 dernières heures, tri décroissant, projection src_ip + timestamp.
R :

```
db.security_logs.find(
  { protocol: "SSH", timestamp: { $gte: new Date(Date.now() -
    86400000) } },
  { src_ip: 1, timestamp: 1, _id: 0 }
).sort({ timestamp: -1 })

```

— **Q2 :** Différence $push vs $addToSet ?
R : $push ajoute même si doublon. $addToSet n'ajoute que si absent.

— **Q3 :** Deux raisons sécurité d'utiliser la projection ?
R : 1) Minimisation des données (pas d'emails/raw_logs exposés). 2) Réduction de la surface d'attaque en cas de compromission de l'API.

9.3 Checkpoint 3

— **Q1 :** Requêtes fréquentes par event_type + timestamp. Quel index composé ?
R :

```
db.security_logs.createIndex({ timestamp: 1, event_type: 1 })

```

— **Q2 :** Quel stage d'aggregation en premier ? Pourquoi ?
R : $match en premier → réduit le volume + exploite les index.

— **Q3 :** Pipeline commençant par $group sans $match — risque ?
R : $group traite TOUS les documents. Sur des millions de logs = consommation massive CPU/RAM → DoS applicatif involontaire.

10 Commandes de nettoyage

```
// Quitter mongosh
quit()

// Arreter et supprimer le conteneur
docker stop mongo-soc
docker rm mongo-soc
```

11 Conclusion

Ce TP a permis de :

- Comprendre la différence entre Embed et Reference en modélisation NoSQL
- Implémenter une base de données SOC-lite fonctionnelle
- Maîtriser les requêtes avancées (filtres, projections, aggregation pipeline)
- Créer et évaluer des indexes pour optimiser les performances
- Appliquer des principes de sécurité data-level (minimisation, masquage PII)